



PROGRAMAÇÃO I

Aula 04

Vinicius Bischoff

O que são Declarações de Switch-case?

As declarações de switch-case em Python são uma estrutura de controle de fluxo que permite que o programa execute diferentes ações com base no valor de uma variável ou expressão.

Embora o Python não tenha uma instrução switch-case nativa, é possível implementar uma funcionalidade semelhante usando uma sequência de declarações if-elif.

Por exemplo, se quisermos executar uma ação diferente com base no valor de uma variável x, podemos fazer o seguinte em Python:

O que são Declarações de Switch-case?

```
if x == 1:  
    # Executa ação 1  
elif x == 2:  
    # Executa ação 2  
elif x == 3:  
    # Executa ação 3  
else:  
    # Executa ação padrão
```

Declarações de switch-case e match-case?

Embora tanto o switch-case quanto o match-case sejam estruturas de controle de fluxo que permitem que um programa execute diferentes ações com base no valor de uma variável ou expressão, existem algumas diferenças importantes entre eles, especialmente em relação à sua implementação em Python.

Declarações de switch-case e match-case?

O switch-case é uma estrutura de controle de fluxo comum em várias linguagens de programação, incluindo C e Java, mas que **não é suportada nativamente** em Python.

Em Python, o switch-case geralmente é implementado usando várias instruções **if-elif-else**, conforme mencionado anteriormente nesta conversa.

Declarações de switch-case e match-case?

O switch-case é uma estrutura de controle de fluxo comum em várias linguagens de programação, incluindo C e Java, mas que **não é suportada nativamente** em Python.

Em Python, o switch-case geralmente é implementado usando várias instruções **if-elif-else**, conforme mencionado anteriormente nesta conversa.

Declarações de switch-case e match-case?

Por outro lado, o match-case é uma nova sintaxe introduzida em Python 3.10, com a PEP 634, que permite realizar uma correspondência de padrões em valores de dados complexos, como listas, tuplas, dicionários e classes.

Declarações de switch-case e match-case?

A sintaxe de padrões estruturais do match-case é mais flexível que o switch-case, permitindo que o programador execute ações com base em padrões personalizados e em estruturas de dados aninhadas.

Exemplo de switch-case em Python usando if-elif-else

```
def operacao(a, b, op):  
    if op == '+':  
        return a + b  
    elif op == '-':  
        return a - b  
    elif op == '*':  
        return a * b  
    elif op == '/':  
        return a / b  
    else:  
        raise ValueError('Operação inválida')  
resultado = operacao(5, 2, '*')  
print(resultado) # Output: 10
```

Exemplo de switch-case em Python (usando if-elif-else):

Exemplo de match-case em Python

```
def operacao(a, b, op):  
    match op:  
        case '+':  
            return a + b  
        case '-':  
            return a - b  
        case '*':  
            return a * b  
        case '/':  
            return a / b  
        case _:  
            raise ValueError('Operação inválida')  
resultado = operacao(5, 2, '*')  
print(resultado)
```

Exemplo de match-case em Python:

Declarações de switch-case e match-case?

Note que no exemplo de match-case, a sintaxe é mais clara e concisa, e o código é mais fácil de ler e manter.

Além disso, a primeira correspondência é encontrada e o código correspondente é executado, enquanto no exemplo de switch-case, todas as condições precisam ser avaliadas até que uma correspondência seja encontrada.

Declarações de switch-case e match-case?

Exemplo de switch-case com ramificações em Python usando if-elif-else

```
def faixa_etaria(idade):  
    if idade < 18:  
        print('Menor de idade')  
    elif idade < 60:  
        print('Adulto')  
    else:  
        print('Idoso')
```

faixa_etaria(25) # Output: Adulto

Declarações de switch-case e match-case?

Exemplo de match-case sem ramificações em Python

```
def faixa_etaria(idade):  
    match idade:  
        case idade < 18:  
            print('Menor de idade')  
        case idade < 60:  
            print('Adulto')  
        case _:   
            print('Idoso')
```

faixa_etaria(25) # Output: Adulto

Funções

- Também conhecidas como métodos ou procedimentos
- Organização do código fonte em blocos padronizados
- Reaproveitamento de códigos executados de forma idêntica em diversas partes do programa

Funções

- Facilita a inclusão de novas funcionalidades, manutenção e a correção de falhas
- Passagem de parâmetros é opcional
- Retorno de resultados é opcional

Estrutura de uma Função

```
def nome_da_funcao(lista de parametros):  
    # Uma ou mais instruções  
    # Utilizando expressões e operadores  
    # Utilizando estruturas de controle  
    # Tanto de seleção quanto repetição  
    return valor
```


Função básica

```
def minha_funcao():  
    print('\nEstou usando funções!\n')
```

Declaração e implementação da função. Ela só será executada quando for chamada

```
# Uma ou mais instruções antes da chamada da função  
minha_funcao()  
  
# Uma ou mais instruções após a chamada da função
```

Chamada da função. Ela é executada nesse momento, podendo ser chamada diversas vezes

Função com retorno

```
def minha_funcao(par1, par2):  
    soma = par1 + par2  
    return soma
```

A função soma os valores dos dois parâmetros e retorna o resultado para o chamador

```
if __name__ == '__main__':  
    v1 = float(input('\nInforme o primeiro número: '))  
    v2 = float(input('Informe o segundo número: '))  
    retorno = minha_funcao(v1, v2)  
    print('\nRetorno da função:', retorno, '\n')
```

Na chamada da função, são passados os dois valores e o retorno é armazenado em uma variável

Exemplos

```
def calcular_media(numeros):  
    soma = sum(numeros)  
    media = soma / len(numeros)  
    return media
```

sum e len são funções internas (built-in) do Python que podem ser usadas para realizar operações matemáticas e obter informações sobre sequências de dados, respectivamente.

A função sum é usada para calcular a soma dos elementos em uma sequência de dados, como uma lista ou uma tupla.

A função len é usada para obter o número de elementos em uma sequência de dados.

```
# Exemplo de uso
numeros = [2, 4, 6, 8, 10]
media = calcular_media(numeros)
print("A média é:", media)
# Output: A média é: 6.0
```

```
def calcular_imc(peso, altura):  
    altura_metros = altura / 100  
    imc = peso / (altura_metros ** 2)  
    return imc
```

Exemplo de uso

```
peso = 75
```

```
altura = 175
```

```
imc = calcular_imc(peso, altura)
```

```
print("O IMC de uma pessoa com peso", peso, "kg e altura", altura,  
      "cm é:", imc)
```

Programas com Menus

- Exibição de uma lista de opções numeradas
- Uma variável para armazenar a escolha do usuário
- Laço para permanecer no programa enquanto a escolha for diferente da opção “Sair”
- Estrutura de seleção para identificar a escolha informada
- Um procedimento por escolha

Menu

```
import os

# Menu com as opções de escolha
def menu():
    os.system('cls' if os.name == 'nt' else 'clear') # Limpa a tela - Win/Linux
    print('\n...: Desenvolvendo programas com menus :...\n')
    print('1 - Item 1')
    print('2 - Item 2')
    print('3 - Item 3')
    print('9 - Sair\n')
    item = input('Escolha uma opção: ')
    return item
```

Opções do Menu

```
# Uma função para cada item do menu
def opcao_1():
    print('\nOpção escolhida: 1\n')

def opcao_2():
    print('\nOpção escolhida: 2\n')

def opcao_3():
    print('\nOpção escolhida: 3\n')
```


Processamento do Menu

```
# Processamento do menu e chamada das funções
escolha = '0'

while(escolha != '9'):
    escolha = menu()
    if escolha == '1':
        opcao_1()
    elif escolha == '2':
        opcao_2()
    elif escolha == '3':
        opcao_3()
    elif escolha == '9':
        print('\nSaindo...\n')
    else:
        print('\nOpção desconhecida!\n')

input('Pressione ENTER para continuar')
```

Obrigado.

